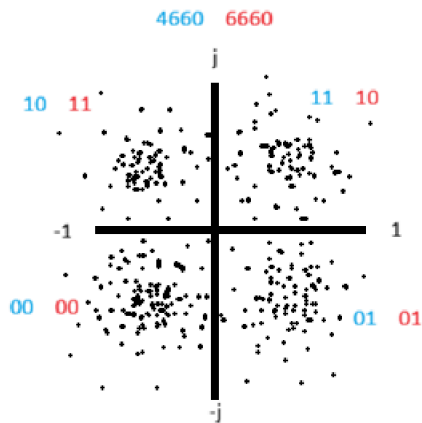


P1) Describe and explain concisely the effect on the BPSK or QPSK signal constellations when the SNR value is increased or decreased. Sketch the constellation of QPSK in complex coordinate plane.

As SNR decreases, the noise variance increases, so the received constellation points are more scattered. In BPSK, this is an issue if this variance goes over the Real 0 midpoint, variance in the imaginary axis makes no difference. In QPSK, crossing either the Real 0 or Imag 0j point crosses into another symbol and causes an error.



P2) Concisely **describe** your decoding methods. Also, include your code as an Appendix in your report.

For BPSK, it's simply $\text{if}(\text{Real} > 0) \text{ bit}=1 \text{ else bit}=0$
 $\text{out} = (\text{Real} >= 0)$

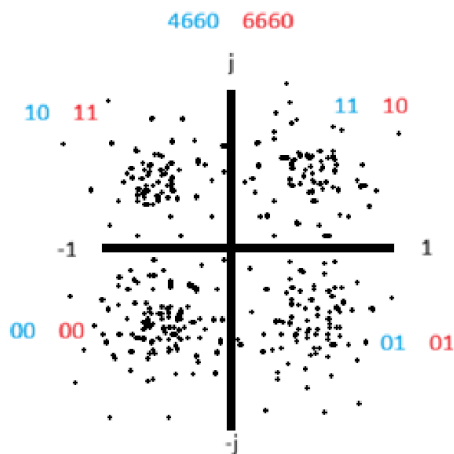
For QPSK 4660, it's low bit is the same as BPSK, and the high bit is the same but imaginary,
 $\text{bit_Q} = (\text{Imag} >= 0)$
 $\text{bit_I} = (\text{Real} >= 0)$
 $\text{out} = (\text{bit_Q} << 1) | \text{bit_I}$

For QPSK 6660, it's just a little bit more complex since the low bit isn't dependent on just one axis, but is actually dependent on if they're signed differently or not. This is achieved in code with a simple NOR of the high bit (which is wholly dependent on the imaginary portion) and the sign of the Real portion

$\text{bit_Q} = (\text{Imag} >= 0)$
 $\text{bit_I} = (\text{bit_Q} \wedge (\text{Real} >= 0))$
 $\text{out} = (\text{bit_Q} << 1) | \text{bit_I}$

P3) **Scratch** the new constellation of QPSK as well. **Answer the question** in Part 2 task 6. (Comparing to the original constellation modulation, which one has lower BER under the same SNR? Explain briefly why this would happen?)

The original 4660 QPSK has lower BER at the same SNR. Both QPSKs have the same SER, but with the 4660 QPSK, a symbol error usually results in only one bit error, while in the 6660 QPSK, for two of the bits, a symbol error has nearly a 50% chance of becoming two bit errors. This is because we put 00 and 11 right next to each other.

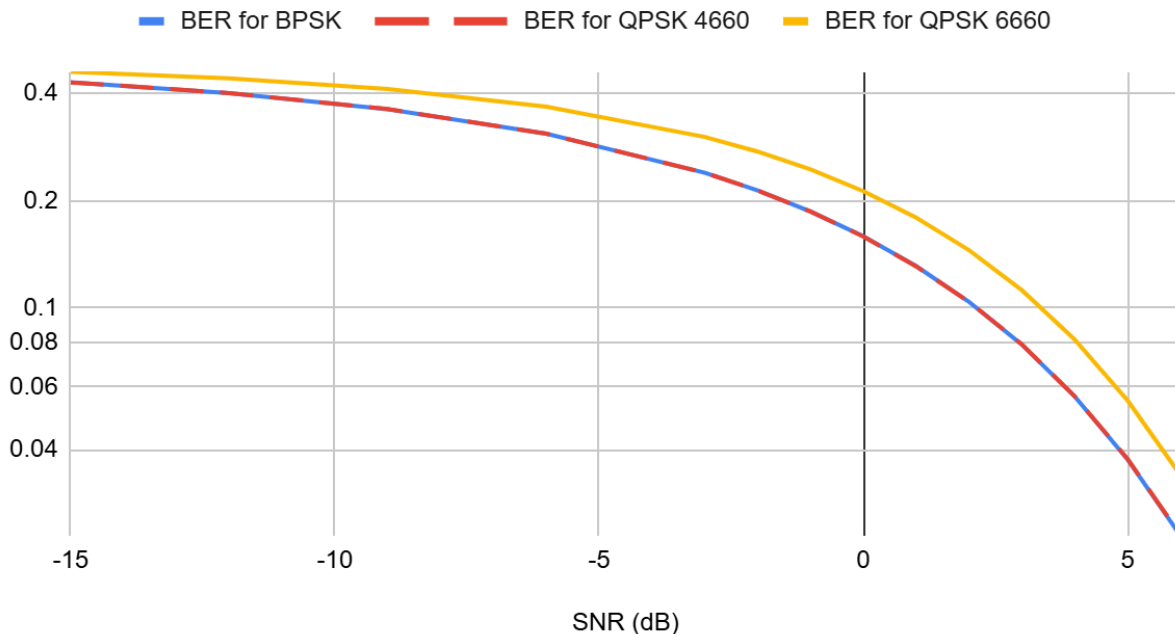


P4) **Record the BER readings** using tables in the Appendix (correct to 3 significant figures) for the SNR values given in the table. Describe and explain the observed trend in a few sentences. BPSK and normal QPSK 4660 have the exact same BER as expected, even though QPSK has a higher SER. QPSK 6660 has the same higher SER, but additionally more bit errors per symbol error, leading to higher BER.

As SNR drops significantly, it's essentially totally random, any bit could be 1 or 0, so they all trend towards BER=0.5 at very low SNR, but have exponentially better results as SNR increases.

P5) **Generate plots from the readings you collect.** Use the '*semilogy*' command if you are using MATLAB, python.matplotlib.semilogy, or manually taking a logarithm to the BER to plot the log(BER)-vs-SNR.

BER for BPSK, BER for QPSK 4660 and BER for QPSK 6660



P6) Determine the SNR values beyond which the symbols can be decoded with great accuracy ($\text{BER} < 0.0001$) for BPSK and QPSK modulations. You can manually type a more precise SNR in the GUI while running the flowgraph. Reading from your plotted graph is also a reasonable method.

From plugging numbers into the simulation:

BPSK: 11.4dB

QPSK 4660: 11.8dB

QPSK 6660: 12.1dB

P7) Should BER vs SNR curve for BPSK and QPSK the same? Why or why not?

For the gray coded 4660 QPSK, yes. While QPSK has two ways of having a symbol error, each of these ways only causes an issue for half the bits in the symbol. Each QPSK symbol is basically just two independent BPSK symbols orthogonal to each other.

Appendix A+B

SNR (dB)	BER for BPSK	BER for QPSK 4660	BER for QPSK 6660
-15	0.42975	0.429475	0.459475
-12	0.40065	0.400775	0.440675
-9	0.36115	0.361375	0.411625
-6	0.30825	0.308225	0.367175
-3	0.23945	0.239475	0.301875
-2	0.21355	0.213575	0.274625
-1	0.18655	0.186375	0.244825
0	0.15875	0.158675	0.212775
1	0.13095	0.130825	0.179125
2	0.10395	0.103975	0.145225
3	0.07885	0.079025	0.112125
4	0.05645	0.056475	0.081475
5	0.03755	0.037675	0.055025
6	0.02295	0.023075	0.033975

```

#BPSK DECODER
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # other base classes are basic_block, decim_block,
interp_block
    def __init__(self): # only default arguments here
        gr.sync_block.__init__(
            self,
            name='BPSK Decoder', # block name
            in_sig=[np.complex64], # input data type
            out_sig=[np.byte] # output data type
        )

    def work(self, input_items, output_items):
        in0 = input_items[0]
        out = output_items[0]
        out[:] = (in0.real >=0).astype(np.uint8)

        return len(out)

```

```

#BPSK BER
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # other base classes are basic_block, decim_block,
interp_block
    def __init__(self, window_size=10000): # only default arguments here
        gr.sync_block.__init__(
            self,
            name='BER for BPSK', # will show up in GRC
            in_sig=[np.byte, np.byte],
            out_sig=[np.float32]
        )

        self.window_size = int(window_size)
        self.window = np.zeros(int(window_size))
        self.win_ptr = 1 % self.window_size

    def work(self, input_items, output_items):
        tx = input_items[0].astype(np.uint8)
        rx = input_items[1].astype(np.uint8)
        out = output_items[0]

        # XOR gives bit differences
        err = tx ^ rx

        # Count number of bit errors in this symbol BPSK 0-1
        biterrors = (err & 0b01)

        for i in range(len(biterrors)):
            # Update circular window
            self.window[self.win_ptr] = biterrors[i]
            self.win_ptr = (self.win_ptr + 1) % self.window_size

            # Compute BER
            out[i] = np.sum(self.window) / (self.window_size)

        # output_items is updated once per function call. It's lazy but fast.
        BER = sum(self.window) / self.window_size
        output_items[0][:] = np.ones(len(input_items[0][:])) * BER

        return len(out)

```

```

#QPSK DECODER 4660
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # other base classes are basic_block, decim_block,
interp_block
    def __init__(self): # only default arguments here
        gr.sync_block.__init__(
            self,
            name='QPSK Decoder', # block name to display
            in_sig=[np.complex64], # input data type
            out_sig=[np.byte] # output data type (similar to char in
C/C++)
        )

    def work(self, input_items, output_items):
        in0 = input_items[0]
        out = output_items[0]
        bit_Q = (in0.imag >= 0).astype(np.uint8)
        # 4660
        bit_I = (in0.real >= 0).astype(np.uint8)
        # 6660
        #bit_I = (bit_Q ^ (in0.real >= 0)).astype(np.uint8)

        out[:] = (bit_Q << 1) | bit_I

        return len(out)

```

```

#QPSK DECODER 6660
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # other base classes are basic_block, decim_block,
interp_block

    def __init__(self): # only default arguments here
        gr.sync_block.__init__(
            self,
            name='QPSK Decoder', # block name to display
            in_sig=[np.complex64], # input data type
            out_sig=[np.byte] # output data type (similar to char in
C/C++)
        )

    def work(self, input_items, output_items):
        #normal 4660
        # -1 1
        #j 10 11
        #-j 00 01
        #6660 additional task
        # -1 1
        #j 11 10
        #-j 00 01
        in0 = input_items[0]
        out = output_items[0]
        bit_Q = (in0.imag >= 0).astype(np.uint8)
        # 4660
        #bit_I = (in0.real >= 0).astype(np.uint8)
        # 6660
        bit_I = (bit_Q ^ (in0.real >= 0)).astype(np.uint8)

        out[:] = (bit_Q << 1) | bit_I

        return len(out)

```



```

#QPSK BER 4660+6660
import numpy as np
from gnuradio import gr

class blk(gr.sync_block): # other base classes are basic_block, decim_block,
interp_block

    def __init__(self, window_size=10000): # only default arguments here
        gr.sync_block.__init__(
            self,
            name='BER for QPSK', # will show up in GRC
            in_sig=[np.byte, np.byte],
            out_sig=[np.float32]
        )

        self.window_size = int(window_size)
        self.window = np.zeros(int(window_size))
        self.win_ptr = 1 % self.window_size

    def work(self, input_items, output_items):

        tx = input_items[0].astype(np.uint8)
        rx = input_items[1].astype(np.uint8)
        out = output_items[0]

        err = tx ^ rx
        # Count number of bit errors in this symbol QPSK 0-2
        biterrors = (err & 0b01) + ((err & 0b10) >> 1)

        for i in range(len(biterrors)):
            # Update circular window
            self.window[self.win_ptr] = biterrors[i]
            self.win_ptr = (self.win_ptr + 1) % self.window_size

            # Compute BER
            out[i] = np.sum(self.window) / (self.window_size)

        # output_items is updated once per function call. It's lazy but fast.
        BER = sum(self.window) / self.window_size
        output_items[0][:] = np.ones(len(input_items[0][:])) * BER

        return len(out)

```